

OmniSense AI Wizard

Master Project Documentation

Agentic Decision Intelligence for Industrial Maintenance

Generated for Technical Review

1. Executive Summary & Problem Statement

Industrial maintenance teams often need to combine equipment alerts, sensor abnormalities, maintenance history, SOPs, manuals, spare-part constraints, and operational priorities before deciding what action to take. This process is frequently manual, time-consuming, and dependent on individual experience.

In highly complex, capital-intensive steel manufacturing environments, delayed or inaccurate troubleshooting leads to catastrophic unplanned downtime, severe production loss, and heightened safety risks.

2. Solution Overview

OmniSense AI Wizard is an evidence-based maintenance decision intelligence system designed to help maintenance engineers move from equipment alerts to actionable maintenance decisions.

Instead of only identifying potential failures, the system evaluates equipment condition, retrieves relevant maintenance knowledge, prioritizes assets, compares maintenance strategies, and recommends the next best action through a transparent and explainable workflow.

3. Key Features & Agentic Decision Intelligence

Telemetry Anomaly Detection: Analyzes equipment telemetry (vibration, temperature, load) using Machine Learning to generate anomaly scores, providing early indications of abnormal behavior.

Knowledge Retrieval (RAG): Retrieves relevant SOPs and manuals using vector search, providing traceable evidence for every recommendation.

Root Cause Analysis: Ranks probable causes (e.g., Bearing wear, Lubrication issues) supported by evidence and confidence scoring.

Maintenance Priority Index (MPI): Prioritizes actions by combining failure probability, safety risk, plant impact, asset criticality, and spare availability.

Strategy Evaluation Engine: Evaluates multiple strategies (e.g., Immediate shutdown vs. Load reduction vs. Planned replacement) based on feasibility.

Natural Language Assistant: Engineers interact via Voice or Text to ask questions like *"What caused the high priority score?"* and get grounded responses.

4. Technical Architecture

High-Level System Architecture

Input / Interfaces	Core Engine (Backend)	Outputs / Actions
Voice/Text (Whisper AI)	FastAPI Backend (Railway)	Structured JSON
Image Uploads	LangGraph Orchestrator	Visual Fault Diagnostics
CSV Data Stream	Telemetry DB & Models	SOP Recommendations
React.js Frontend (AWS Amplify)	FAISS Vector DB	Generated Tickets

LangGraph Multi-Agent Workflow

The system orchestrates 8 specialized agents using a LangGraph Directed Acyclic Graph (DAG).

- **Orchestrator:** Classify_Intent, Route_to_Expert
- **Vision_Agent:** Analyzes visual faults
- **RAG_Agent:** Search_FAISS, Synthesize_Context
- **Anomaly_Agent:** Queries sensors
- **Diagnostic_Agent:** Troubleshoots logic
- **Report_Generator:** Generate_Ticket, Return JSON

5. Model Selection Rationale

- **AI Orchestration:** LangGraph

Why? Unlike standard linear LangChain pipelines, LangGraph allows for stateful, multi-actor applications with cyclic routing. It lets our agents "think, act, observe, and correct" their logic before replying.

- **Fast Reasoning LLM: Groq (Mixtral-8x7b & Llama 3)**

Why? Maintenance in a steel plant requires instant answers. Groq provides LPU (Language Processing Unit) execution resulting in ultra-low latency (>500 tokens/sec), ensuring engineers get real-time answers.

- **Multimodal Vision: Google Gemini 1.5 Flash (Fallback: Mistral Pixtral)**

Why? Gemini 1.5 possesses state-of-the-art vision capabilities for identifying physical asset damage (e.g., spalling on bearings, cracked pipes) from low-quality plant floor photos.

- **Voice-to-Text: OpenAI Whisper (Base Model)**

Why? Heavy industry environments require hands-free operation. Whisper is highly robust against background noise (like machinery roar) and can run locally without exposing sensitive audio to external APIs.

- **Remaining Useful Life (RUL): XGBoost Regressor**

Why? XGBoost is highly interpretable, handles tabular sensor/delay log data exceptionally well, and prevents overfitting compared to deep learning models for structured industrial data.

- **Anomaly Detection: Isolation Forest**

Why? Equipment rarely fails, making labeled "failure" data scarce. Isolation Forest is an unsupervised algorithm that perfectly detects novel anomalies in telemetry without needing vast amounts of labeled failure data.

- **Vector Database: FAISS (Facebook AI Similarity Search)**

Why? It runs locally in-memory, requiring no external cloud database. It provides millisecond retrieval of complex SOP chunks using HuggingFace sentence transformers.

6. Business Impact & Value Proposition

- **Reduce Unplanned Downtime:** Early warnings and accurate RUL prediction prevent sudden catastrophic failures.
- **Improve Response Time:** Engineers no longer spend hours hunting through physical binders; RAG surfaces the exact repair step in seconds.

- **Increase Diagnostic Accuracy:** Combining visual AI, sensor data, and SOPs standardizes repair quality across shifts, regardless of engineer experience.
- **Improve Spare-Part Planning:** Linking diagnostics with procurement constraints prevents taking a machine offline when parts aren't available.
- **Decision Transparency:** Eliminates "Black-box AI" by citing exact PDF pages and sensor readings for every decision.

7. Future Scope & Scalability

With real plant integration, the OmniSense architecture can seamlessly scale:

- **IoT Integration:** Direct connection to SCADA systems and PLCs for live, continuous telemetry ingestion instead of CSV uploads.
- **Enterprise Integration:** Connecting the Graph agents directly to CMMS (Computerized Maintenance Management Systems) like SAP PM or IBM Maximo for automated work order generation.
- **Edge Deployment:** Running SLMs (Small Language Models) securely on localized servers within the plant to eliminate internet dependency and enhance data privacy.
- **AR / Smart Glasses:** Adapting the UI for augmented reality headsets, projecting step-by-step repair guides directly into the engineer's field of view.

8. Project Directory Structure

```

steelmind-ai-wizard/
├── frontend/                                # React.js UI (AWS Amplify)
│   ├── src/
│   │   ├── components/                    # UI Components (Dashboard, Chat, Tickets)
│   │   ├── api.js                        # API configurations
│   │   └── index.css                      # Tailwind CSS styles
│   └── package.json                      # NPM dependencies
├── src/                                    # Python Backend (FastAPI / Railway)
│   ├── agents/                          # Specialized AI Agents
│   ├── data/                            # Historical logs & SQLite DB
│   ├── graph/                          # LangGraph DAG config
│   ├── knowledge_base/                  # PDF Manuals & FAISS Index
│   ├── models/                         # Pickled ML models
│   └── utils/                          # Whisper Audio & Prompts
├── main.py                              # Backend Entrypoint (uvicorn)
├── requirements.txt                     # Python Dependencies
└── amplify.yml                         # CI/CD Deployment Config

```

9. Instructions to Run

Frontend Setup (React):

1. Navigate to the frontend directory: `cd frontend`
2. Install dependencies: `npm install`
3. Create a `.env` file containing the production API URL:
`VITE_API_URL=https://steelmind-ai-wizard-production.up.railway.app`
4. Start the development server locally: `npm run dev`

Note: The frontend is also continuously deployed and available live on AWS Amplify.

Backend Setup (FastAPI & Python):

1. Navigate to the project root directory.
2. Create and activate a Python virtual environment:
`python -m venv venv`
`source venv/bin/activate` (Mac/Linux) or `venv\Scripts\activate` (Windows)
3. Install required packages: `pip install -r requirements.txt`
4. Create a `.env` file in the root with your API Keys:
`GROQ_API_KEY=your_key_here`
`GOOGLE_API_KEY=your_key_here`
`MISTRAL_API_KEY=your_key_here`
5. Start the FastAPI server: `uvicorn main:app --host 0.0.0.0 --port 8000`

Note: The backend is also continuously deployed and available live on Railway.